Homework 3: Baseball Data

Eli Crowder

2026-01-29

1. What line of code did you have to run in R Studio before you could successfully run the code from Section 3.6 on your own computer? Why did you need to do that? Do you need to do it again the next time you want to run the code from Section 3.6 on your computer?

I had to run `install.packages("Lahman")` in the Console and I had to include and run `library(ggplot2)`, `library(tidyverse)`, and `library(Lahman)` in the actual document. You do not need to install it each time but you need to run the library function each time you open the document.

2. How many variables are in the original `Batting` dataset? Name three or four of them and say what they represent. (Run `?Batting` or `?Lahman::Batting` in your Console to find out more about the other variables. Do **not** include any `?` commands in your .qmd documents because the `?` command is meant only for an interactive environment, and the .qmd file will not render.)

There are 22 variables in the `Batting` dataset, including `playerID` (player ID code), `yearID` (year), `stint` (the players stint or order of appearances within a season), and `G` (games: number of games in which a player played) to name a few.

3. What variables (columns) are in the `batters` dataset that Section 3.6 creates from the `Batting` dataset?

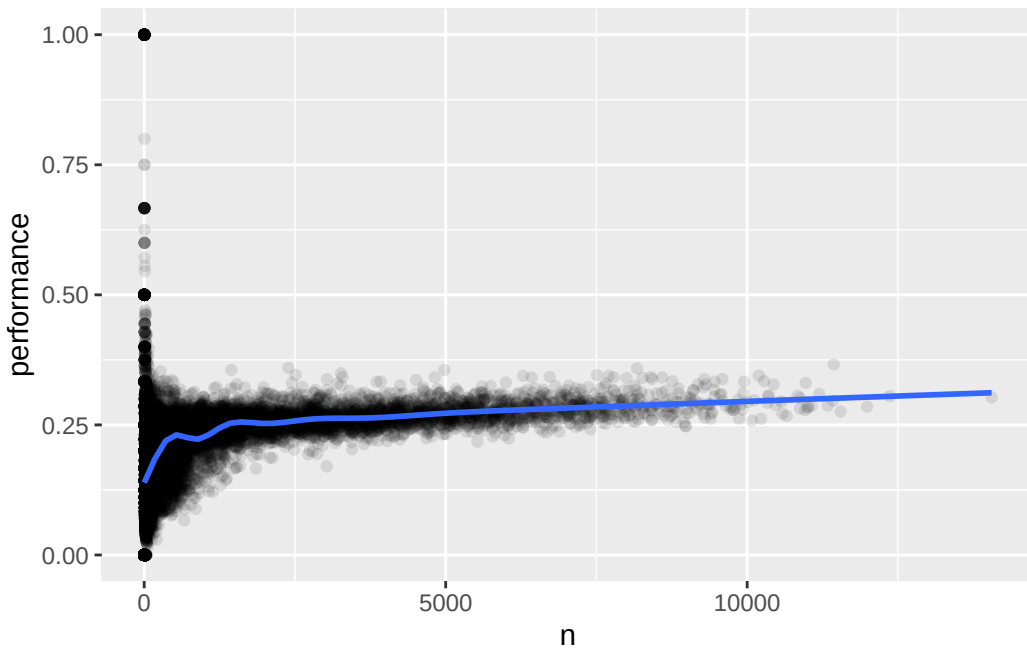
There are three variables/columns. They are `playerID`, `preformance` (hits divided by at bats), and `n` (at bats).

4. What does each row of `batters` represent?

Each row of `batters` represents one individual player.

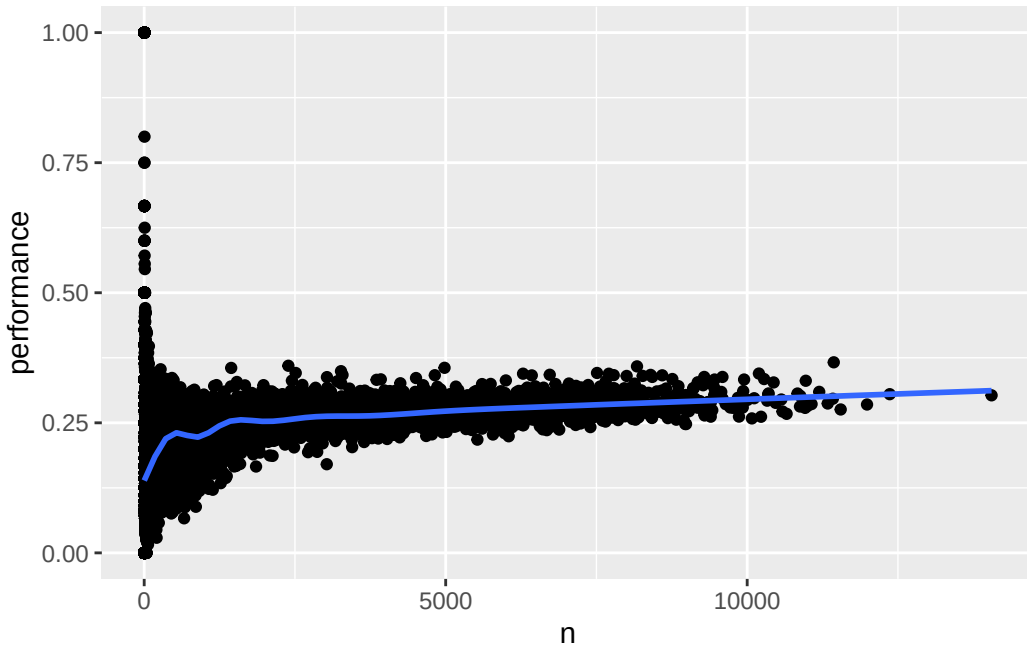
5. Take the second code chunk in Section 3.6 and remove the line that drops rows with n less than or equal to 100. Show the resulting code and the new plot you get. (Note: the only code you should show in your homework pdf is the code for this plot, but you will need to run some other code first in order for that code to run in your .qmd file.)

```
batters |>
  ggplot(aes(x = n, y = performance)) +
  geom_point(alpha = 1 / 10) +
  geom_smooth(se = FALSE)
```



6. Let's figure out what the `alpha` argument does. Modify the second code chunk in Section 3.6 by removing the `alpha` argument so that there is nothing inside the parentheses of `geom_point`. Show the resulting code and the new plot you get. Based on this and on the results of `?alpha` (run `?alpha` in the Console but don't include `?alpha` in your .qmd document), what does `alpha` control about the plot?

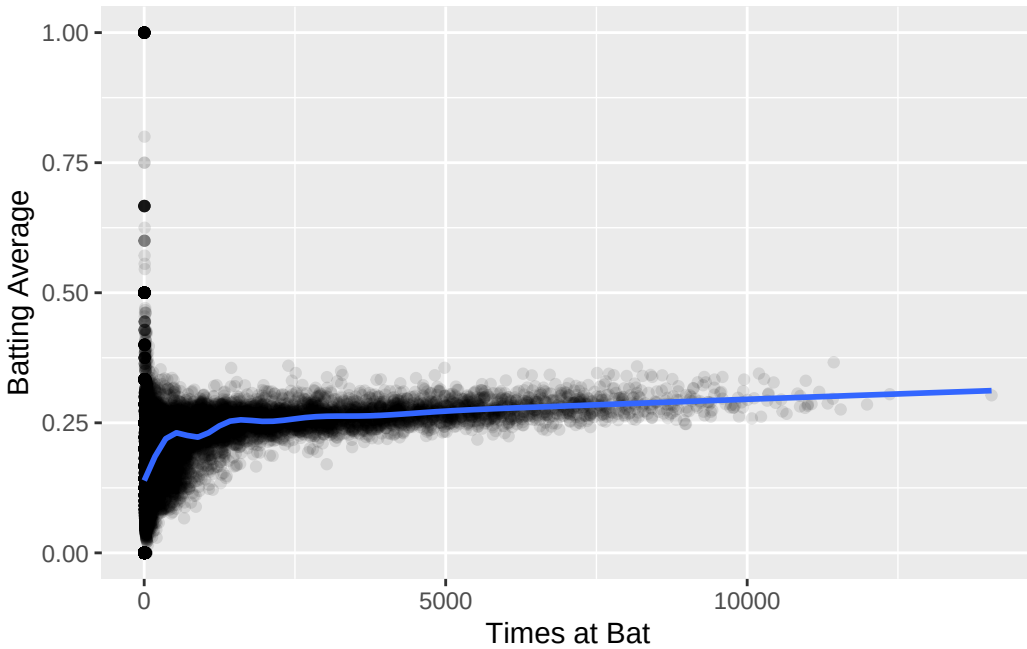
```
batters |>
  ggplot(aes(x = n, y = performance)) +
  geom_point() +
  geom_smooth(se = FALSE)
```



The `alpha` argument allows the user to modify color transparency of the points. Overlapping points become lighter, making less dense regions less dark and more dense regions very dark.

7. Adjust the second code chunk in Section 3.6 to give the plot more formal and informative axis labels, such as "Batting Average" for the vertical axis and "Times at Bat" on the horizontal axis.

```
batters |>
  ggplot(aes(x = n, y = performance)) +
  geom_point(alpha = 1 / 10) +
  geom_smooth(se = FALSE) +
  labs(x = "Times at Bat", y = "Batting Average")
```

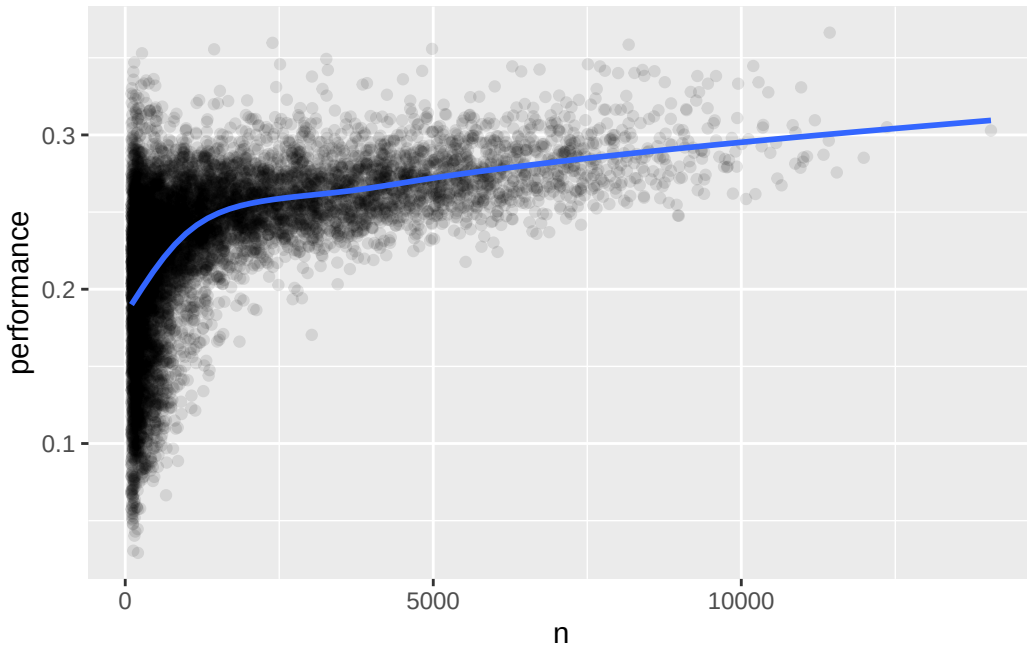


8. Rewrite the first code chunk in Section 3.6 to use the `.by` argument of `summarize` instead of using both the `group_by` and `summarize` functions.

```
batters <- Lahman::Batting |>
  summarize(
    .by = playerID,
    performance = sum(H, na.rm = TRUE) / sum(AB, na.rm = TRUE),
    n = sum(AB, na.rm = TRUE)
  )
batters
```

9. Rewrite the second code chunk in Section 3.6 into two separate chunks of code that result in the same final plot. The first code chunk modifies `batters` for plotting by keeping only rows with `n > 100`, and the second chunk plots the new `batters` dataset. As a result of your modified code, the `batters` object itself should now only have values of `n` greater than 100.

```
batters <- batters |>
  filter(n > 100)
batters |>
  ggplot(aes(x = n, y = performance)) +
  geom_point(alpha = 1 / 10) +
  geom_smooth(se = FALSE)
```



10. In the spirit of the first code chunk in Section 3.6, compute a different summary statistic or two related ones from the `Batting` dataset. Briefly summarize in text what you computed. Make sure you update the name of your new dataset from `batters` to something that makes sense for the summary statistic(s) you computed.

```
heavy_hitters <- Lahman::Batting |>
  group_by(playerID) |>
  summarise(
    total_at_bat = sum(AB, na.rm = TRUE),
    total_home_runs = sum(HR, na.rm = TRUE),
    efficiency = total_at_bat / total_home_runs
  ) |>
  filter(total_home_runs > 20)

heavy_hitters |>
  arrange(efficiency)

heavy_hitters
```

I grouped the `Batting` data by player and calculated each player's total at-bats, total home runs, and an efficiency measure defined as at-bats per home run, keeping only players with more than 20 home runs. I then sorted players by this efficiency to compare how frequently different hitters produce home runs.

11. Read Sections 4.2–4.4 of R4DS2e, then apply the style tips there on spaces, line breaks and indentation to improve the style of the code chunk in Section 4.6, Exercise 1.

```
flights |>
  filter(dest == "IAH") |>
  group_by(year, month, day) |>
  summarize(n = n(),
  delay = mean(arr_delay, na.rm = TRUE)) |>
  filter(n > 10)

flights |>
  filter(carrier == "UA",
  dest %in% c("IAH", "HOU"),
  sched_dep_time > 0900, sched_arr_time < 2000) |>
  group_by(flight) |>
  summarize(delay = mean(arr_delay, na.rm = TRUE),
  cancelled = sum(is.na(arr_delay)),
  n = n()) |>
  filter(n > 10)
```